MA374 LAB 02 Report

230123077 Rehan Sherawat

January 18, 2026

1 Problem 1: Standard European Options

1.1 Methodology

We implemented a generalized binomial tree model to price European Call and Put options. The model was tested using two specific parameter sets to verify robustness:

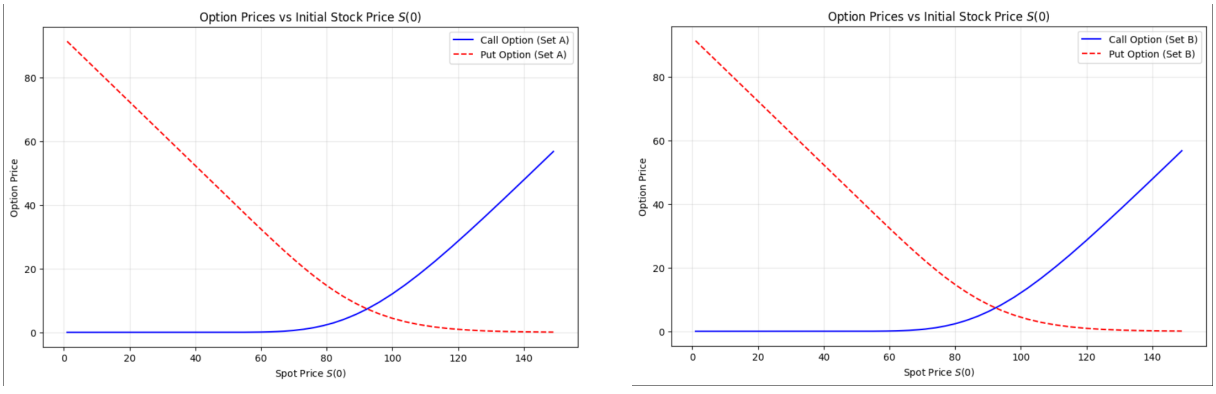
- **Set A:** Standard Cox-Ross-Rubinstein parameters.
- **Set B:** Parameters adjusted for continuous compounding ($u = e^{\sigma\sqrt{\Delta t}+(r-0.5\sigma^2)\Delta t}$).

We analyzed the sensitivity of the option prices to Spot Price (S_0), Strike Price (K), Volatility (σ), and Risk-free Rate (r), as well as the convergence of the price with respect to the number of time steps (M).

1.2 Results and Analysis

1.2.1 Sensitivity to Spot Price (S_0)

As illustrated below, Call option prices (blue) increase monotonically with the initial stock price, while Put option prices (red dashed) decrease. The relationship is non-linear (convex).

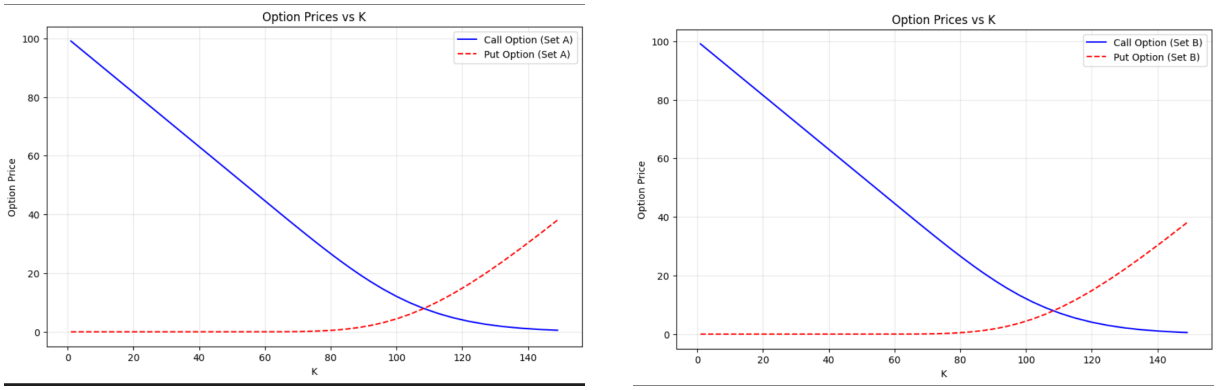


(a) Set A (b) Set B

Figure 1: Option Prices vs Initial Spot Price $S(0)$

1.2.2 Sensitivity to Strike Price (K)

Call option values decrease as the Strike Price K increases, as the option becomes further out-of-the-money. Conversely, Put option values increase as K increases.



(a) Set A

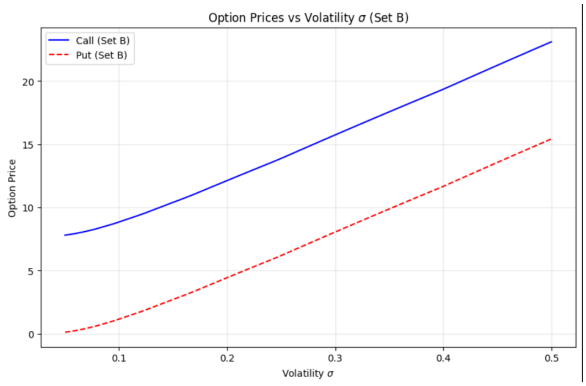
(b) Set B

Figure 2: Option Prices vs Strike Price K

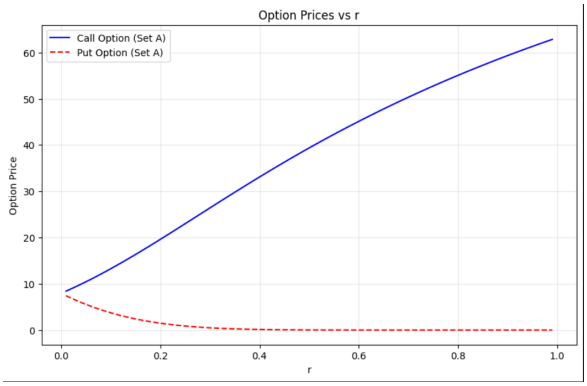
1.2.3 Sensitivity to Market Parameters (σ, r)

Volatility (σ): Higher volatility increases the value of both Calls and Puts (see Figures 3a, 3b) due to the higher probability of extreme price movements (vega is positive).
Interest Rate (r): Higher rates increase Call values and decrease Put values (see Figures 3c, 3d) due to the time value of money affecting the present value of the strike price.

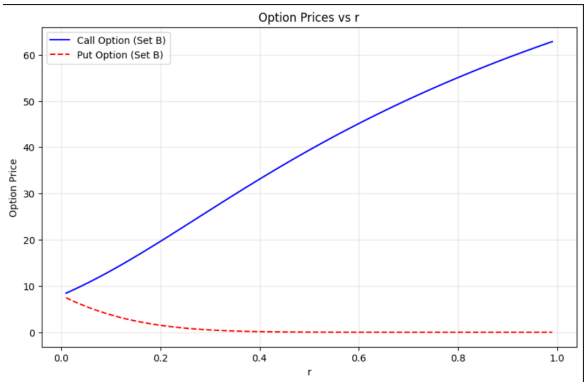
(a) Sensitivity to Volatility (Set A)



(b) Sensitivity to Volatility (Set B)



(c) Sensitivity to Interest Rate (Set A)

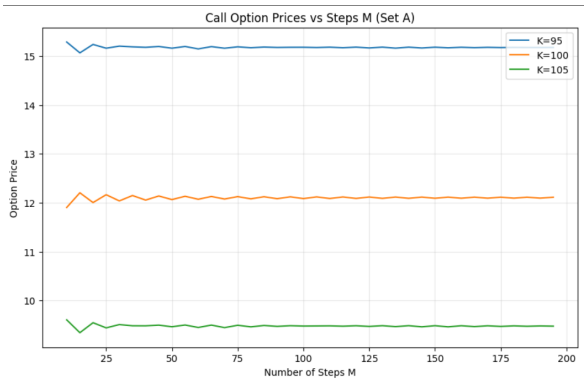


(d) Sensitivity to Interest Rate (Set B)

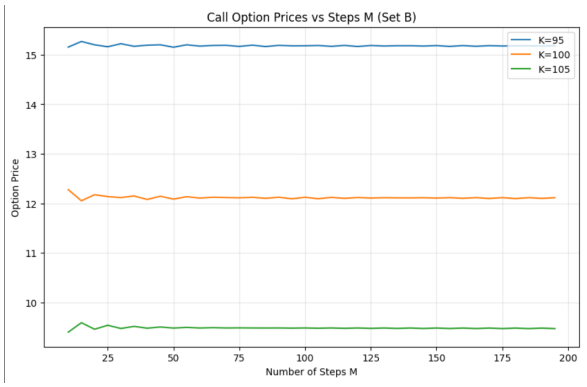
Figure 3: Sensitivity to Volatility and Interest Rates

1.2.4 Convergence with Steps (M)

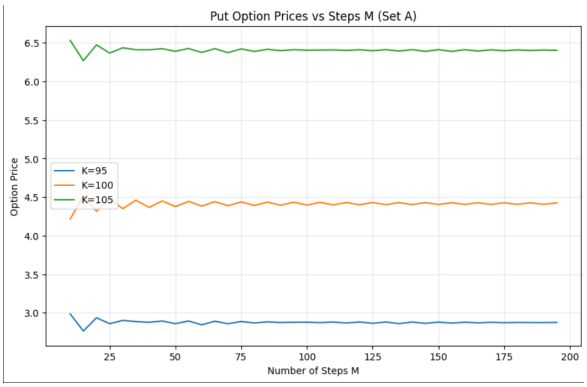
The "sawtooth" pattern in the convergence plots below is a characteristic artifact of the binomial model's discrete nature. As M increases, the price oscillates and converges toward the theoretical Black-Scholes value.



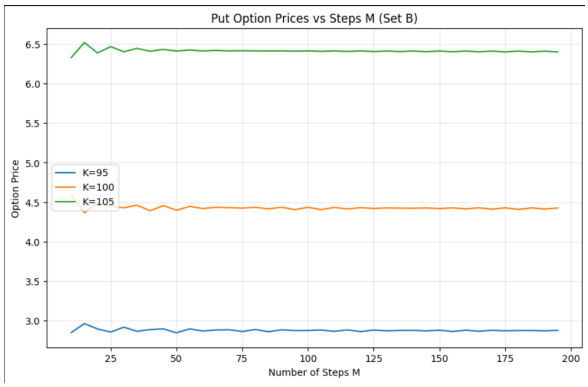
(a) Call Convergence (Set A)



(b) Call Convergence (Set B)



(c) Put Convergence (Set A)



(d) Put Convergence (Set B)

Figure 4: Convergence of Option Price vs Number of Steps M

2 Problem 2: Lookback Options (Basic Algorithm)

2.1 Part A: Pricing Results

We implemented the Basic Lookback Algorithm, tracking the state tuple (S_t, S_{max}). The calculated initial prices for the Lookback Put ($V = S_{max} - S_T$) are:

Time Steps (M)	**Option Price**
5	9.1193
10	10.0806
25	11.0035
50	11.5109

Table 1: Lookback Option Prices for varying M

2.2 Part B: Analysis of Convergence

Observation: As M increases, the option price tends to increase.
Reasoning: Lookback options depend on the extrema (maximum) of the price path. A model with more time steps (higher M) samples the asset price more frequently. This higher sampling frequency increases the probability of capturing transient price peaks

that might occur between the coarser time steps of a low-M model. Since the payoff is $S_{max} - S_T$, a higher recorded S_{max} leads to a higher expected payoff.

2.3 Part C: Intermediate Tree Values ($M = 5$)

Below is the tabulated data for the tree states at $M = 5$. Note that multiple nodes may exist at the same time step if the path histories (Maximums) differ.

Table 2: Lookback Tree Values ($M = 5$)

Time	**Price (S_t)**	**Max (S_{max})**	**Option Value**
0	100.0000	100.0000	9.1193
1	110.6767	110.6767	9.0280
1	90.3548	100.0000	9.5048
2	122.4932	122.4932	8.5481
2	100.0000	110.6767	9.7991
2	100.0000	100.0000	7.1479
2	81.6380	100.0000	12.1687
3	135.5714	135.5714	7.4168
3	110.6767	122.4932	9.9553
3	110.6767	113.3650	6.2019
3	110.6767	110.6767	13.7129
3	90.3548	113.3650	6.2019
3	90.3548	102.4290	8.3246
3	90.3548	100.0000	7.1484
3	73.7629	100.0000	17.5821
4	150.0459	150.0459	5.5016
4	122.4932	135.5714	9.5714
4	122.4932	125.4686	4.6005
4	122.4932	122.4932	15.6319
4	100.0000	125.4686	4.6005
4	100.0000	113.3650	8.0036
4	100.0000	110.6767	6.6808
4	100.0000	110.6767	21.1881
4	81.6380	125.4686	4.6005
4	81.6380	113.3650	8.0036
4	81.6380	104.9171	3.8469
4	81.6380	102.4290	13.0714
4	81.6380	104.9171	3.8469
4	81.6380	100.0000	10.6809
4	66.6457	100.0000	25.0512
5	166.0657	166.0657	0.0500
5	135.5714	150.0459	11.1814
5	135.5714	138.8645	0.0500
5	135.5714	135.5714	0.0500
5	110.6767	138.8645	19.4527
5	110.6767	138.8645	0.0500

Time	Price (S_t)	Max (S_{max})	Option Value
5	110.6767	125.4686	9.3500
5	110.6767	122.4932	6.3745
5	110.6767	122.4932	25.3946
5	90.3548	138.8645	0.0500
5	90.3548	125.4686	9.3500
5	90.3548	116.1187	0.0500
5	90.3548	113.3650	16.2664
5	90.3548	116.1187	0.0500
5	90.3548	110.6767	13.5780
5	90.3548	110.6767	13.5780
5	73.7629	110.6767	29.4826
5	73.7629	138.8645	0.0500
5	73.7629	125.4686	9.3500
5	73.7629	116.1187	0.0500
5	73.7629	113.3650	16.2664
5	73.7629	116.1187	0.0500
5	73.7629	104.9171	7.8184
5	73.7629	102.4290	5.3304
5	73.7629	102.4290	21.2350
5	60.2889	116.1187	0.0500
5	60.2889	104.9171	7.8184
5	60.2889	100.0000	2.9014
5	60.2889	100.0000	18.8060
5	60.2889	100.0000	2.9014
5	60.2889	100.0000	18.8060
5	60.2889	100.0000	32.1054

3 Problem 3: Efficient Algorithm Comparison

3.1 Methodology

We compared the Basic Algorithm (which tracks absolute states (S, S_{max})) against an Efficient Algorithm based on similarity reduction. The Efficient Algorithm tracks the ratio $R_t = S_{max}/S_t$, reducing the dimensionality of the state space from $O(M^3)$ to approximately $O(M^2)$.

3.2 Performance Results

We executed a runtime comparison for increasing steps M.

Steps (M)	**Basic Algo (s)**	**Efficient Algo (s)**	**Difference**
10	0.0033	0.0044	+0.0011
20	0.0178	0.0229	+0.0050
40	0.8001	0.3316	-0.4685
60	15.8142	1.6226	-14.1916
80	120.6496	6.3812	-114.2684

Table 3: Algorithm Runtime Comparison

3.3 Conclusion on Efficiency

The results demonstrate a clear "Crossover Point" between $M = 20$ and $M = 40$.

- For small M (< 20), the Basic algorithm is marginally faster due to lower overhead.
- As M increases beyond 40, the Basic algorithm's runtime grows cubically, becoming impractical (taking nearly 2 minutes for $M = 80$).
- The Efficient algorithm scales significantly better, handling $M = 80$ in just 6 seconds. This confirms that the dimension reduction technique is essential for pricing path-dependent options with high precision.